ASSBI Platform

AI kuzatuv va Business Intelligence loyihasi boʻyicha toʻliq qoʻllanma

Kod xaritasi • Dashboard/API bogʻlanishi • YOLO detection • Fine-tuning • Deploy

Maqsad

Bu qoʻllanma ogʻzaki himoya va texnik tushuntirish uchun yozildi. Unda loyiha papkalari, asosiy kodlar, dashboardlar qaysi API'dan ma'lumot olishi, YOLO/OpenCV/FastAPI/React kabi kutubxonalar nima uchun tanlangani va fine-tuning qanday ishlashi tushuntiriladi.

Boʻlimlar

- Loyiha nima qiladi va umumiy arxitektura
- Papka va fayl xaritasi
- Backend API endpointlari
- Frontend dashboardlar va qaysi data manbalarini ishlatishi
- Detection pipeline: RTSP, YouTube, MP4, YOLO, OpenCV
- Fine-tuning nima va bu loyihada qayerda ishlatiladi
- Kutubxonalar tanlovi sabablari
- Ishga tushirish, deploy va troubleshooting

1. Loyiha haqida qisqa tushuncha

ASSBI Platform real vaqt rejimida kameralar va video manbalardan kadr olib, AI detection orqali odamlar, transport, telefon, noutbuk va boshqa obyektlarni aniqlaydi. Natijalar backend API orqali dashboardlarga uzatiladi. Platforma monitoring, risk scoring, hisobot, chatbot, fine-tuning va governance bo‘limlarini birlashtiradi.

Qism	Vazifasi
Frontend	React/Vite dashboard: live kuzatuv, analytics, reports, settings, fine-tuning, chatbot.
Backend API	FastAPI: auth, camera CRUD, analytics, reports, stream/frame endpointlari, chatbot va fine-tuning API.
Detector	OpenCV + Ultralytics YOLO: video/kamera kadrlarini o‘qish, object detection, statistikani databasega yozish.
Relay	Mac/local detector natijasini AWS API’ga yuboradi: /api/ingest/frame endpointi orqali frame va metrikalar boradi.
Database/files	SQLite/csv/json/files: analytics, camera config, frames, reports, model settings.

Muhim arxitektura g‘oyasi

AWS server frontend va API’ni host qiladi. RTSP kamera lokal 192.168.x.x tarmoqda bo‘lsa, uni AWS bevosita ko‘ra olmaydi. Shuning uchun local Mac relay/detector kamera bilan bir tarmoqda ishlaydi va natijani AWS API’ga yuboradi.

2. Project papkalari va qayerda nima bor

Path	Nima uchun kerak
app/api_server.py	Asosiy FastAPI backend. Auth, camera API, analytics, report export, frame/stream, fine-tuning, chatbot shu faylda.
app/main_detector.py	Asosiy YOLO detector. RTSP/YouTube/MP4/webcam manbasidan kadr olib detection qiladi, frame va metrics saqlaydi.
app/frame_grabber.py	YouTube stream uchun original/detection frame grab qilish va overlay chizish.
app/auto_relay_manager.py	API’dagi kameralarni kuzatib, kerakli detector/frame_grabber/relay processlarni avtomatik start/stop qiladi.
app/relay_camera_to_api.py	Local frame va analytics natijasini AWS/FastAPI endpointiga upload qiladi.
app/database.py	SQLite schema va insert/select helperlar: sessions, minute analytics, incidents, audit.
app/modules/*.py	Analytics, privacy blur, suspicious behavior, heatmap, trails, report kabi yordamchi modullar.
frontend/src/app/App.tsx	Router va login/auth flow. Page componentlar lazy load qilingan.
frontend/src/app/components/LoginScreen.tsx	Login sahifasi. User Role select, email/password login.
frontend/src/app/components/pages/*.tsx	Har bir dashboard sahifasi: Live, Reports, Settings, FineTuning va boshqalar.
frontend/src/app/lib/config.ts	Frontend API base URL sozlamasi.
scripts/*.py	Dataset yig‘ish, autolabel, preview, YOLO training kabi fine-tuning yordamchi scriptlari.
docs/*.md	Fine-tuning va Roboflow ishlatmasdan training bo‘yicha mavjud markdown qo‘llanmalar.
Dockerfile.api, frontend/Dockerfile, docker-compose.yml	AWS/VPS deploy uchun container konfiguratsiyasi.

3. Backend API endpointlari

Frontend dashboardlar hammasi FastAPI serverga HTTP request yuboradi. Muhim endpointlar quyidagilar:

Endpoint	Method	Vazifasi
/api/auth/login	POST	Email/password/role bilan login, token yaratadi.
/api/auth/me	GET	Joriy foydalanuvchini tekshiradi.
/api/summary	GET	Dashboard KPI: live people, visitors, risk, FPS, quality va umumiy statistika.
/api/cameras	GET/POST	Kamera ro‘yxati va yangi kamera qo‘shish.
/api/cameras/upload-video	POST	MP4/MOV kabi lokal video faylni upload qilib local-video camera sifatida qo‘shish.
/api/cameras/{id}/start	POST	Tanlangan kamera detector processini start qilish.
/api/cameras/{id}/stop	POST	Tanlangan kamera detector processini stop qilish.
/api/ingest/frame	POST	Relay detector natijasini API’ga yuboradi: frame, people, risk, fps, quality, object counts.
/api/frame/{camera_id}	GET	Eng oxirgi JPEG frame. Snapshot/preview uchun.
/api/stream/{camera_id}	GET	MJPEG stream. Live detection panelda ishlatiladi.
/api/snapshot/{camera_id}	GET	Kamera snapshot faylini ochish/yuklash.
/api/analytics	GET	Vaqt bo‘yicha analytics records.
/api/incidents	GET/PATCH	Incidentlar va workflow statuslari.
/api/predictive	GET	Forecast va trend hisob-kitoblari.
/api/reports/*	GET	Excel/CSV/PDF hisobot exportlari.
/api/fine-tuning/status	GET	Model registry va aktiv model holati.
/api/fine-tuning/model	POST	best.pt kabi YOLO model yuklash.
/api/fine-tuning/select	POST	Aktiv detection modelni tanlash.
/api/fine-tuning/test-image	POST	Rasm upload qilib modelni test qilish.
/api/chat	POST	Chatbot: operational fallback + OpenAI API key bo‘lsa free-form javoblar.

4. Dashboardlar qaysi API’dan data oladi

Frontend sahifa	Fayl	Asosiy API manbalari
Dashboard Overview	DashboardOverview.tsx	/api/summary, /api/analytics, /api/incidents
Live kuzatuv	LiveSurveillance.tsx	/api/cameras, /api/stream/{id}, /api/frame/{id}, /api/snapshot/{id}, /api/reports/analytics/excel
Odamlar analitikasi	CrowdAnalytics.tsx	/api/summary, /api/cameras, /api/reports/analytics/excel
Object Detection	ObjectDetection.tsx	/api/summary, /api/cameras, /api/incidents
Anomaliya detection	AnomalyDetection.tsx	/api/summary, /api/incidents, /api/cameras
Prognoz analitika	PredictiveAnalytics.tsx	/api/summary, /api/cameras, /api/predictive, /api/reports/forecast/excel
Hisobotlar	Reports.tsx	/api/summary, /api/cameras, /api/incidents, /api/reports/*
Baholash va nazorat	GovernanceEvaluation.tsx	/api/evaluation, /api/pipeline,

		/api/compliance, /api/visitors, /api/audit
Fine-tuning	FineTuning.tsx	/api/fine-tuning/status, /api/fine-tuning/model, /api/fine-tuning/select, /api/fine-tuning/test-image
AI Chatbot	AIChatbot.tsx	/api/summary, /api/chat
Sozlamalar	Settings.tsx	/api/cameras, /api/cameras/upload-video, /api/thresholds, /api/settings

Dashboard data oqimi

Detector kadrlarni o‘qiydi -> YOLO natija chiqaradi -> database/frame fayllar yangilanadi -> FastAPI /api/cameras va /api/summary data beradi -> React dashboard 2-3 sekund oralig‘ida polling qilib ekranni yangilaydi.

5. Detection pipeline qanday ishlaydi

1. Kamera yoki video manba qo‘shiladi: YouTube URL, RTSP URL, MP4 upload yoki webcam index.
2. auto_relay_manager.py kamera ro‘yxatini /api/relay/cameras orqali o‘qiydi va mos processlarni start qiladi.
3. main_detector.py OpenCV orqali frame oladi. YouTube bo‘lsa yt-dlp yordamida haqiqiy stream URL olinadi.
4. Frame resize qilinadi va YOLO modelga beriladi: person, car, laptop, phone va boshqa classlar aniqlanadi.
5. Noto‘g‘ri object sanashni kamaytirish uchun odamga juda yopishgan kichik object boxlar filter qilinadi.
6. Risk score active people, object/vehicle count, zone peak va thresholdlar asosida hisoblanadi.
7. Annotated frame frames/ papkasiga yoziladi, latest boxes JSON saqlanadi, analytics databasega yoziladi.
8. relay_camera_to_api.py latest frame/metricsni /api/ingest/frame endpointiga upload qiladi.
9. Frontend /api/stream/{camera_id} yoki /api/frame/{camera_id} orqali natijani ko‘rsatadi.

Source turi	Qanday ishlaydi	Eslatma
RTSP	cv2.VideoCapture(rtsp_url, CAP_FFMPEG) orqali kamera stream o‘qiladi.	Kamera va detector bir LAN/Wi-Fi’da bo‘lishi kerak. AWS 192.168.x.x IP’ni ko‘ra olmaydi.
YouTube live/video	yt-dlp haqiqiy playable stream URL oladi, OpenCV o‘qiydi.	YouTube cookie/POT provider kerak bo‘lishi mumkin. Detection original embeddan alohida ishlaydi.
MP4/MOV upload	Fayl serverga upload bo‘ladi, detector local video sifatida frame-by-frame o‘qiydi.	Live bo‘lmagan video detection test uchun qulay.
Webcam	OpenCV camera index orqali o‘qiydi.	Mac/local machine’da ishlatish osonroq.

6. Nima uchun YOLO va boshqa kutubxonalar tanlandi

Kutubxona/texnologiya	Nima uchun ishlatildi
Ultralytics YOLO	Real-time object detection uchun tez, tayyor pretrained modellari bor, fine-tuning oson, .pt modelni to‘g‘ridan-to‘g‘ri yuklab ishlatish mumkin.
OpenCV	RTSP/MP4/webcam frame o‘qish, resize, JPEG yozish, rectangle/text chizish va low-latency video pipeline uchun eng amaliy kutubxona.
yt-dlp	YouTube sahifasidan bevosita video fayl emas, real stream URL olish uchun kerak. Oddiy iframe detection bermaydi, chunki YOLOga frame kerak.
FastAPI	Python AI backend bilan tabiiy integratsiya, tez API yozish, UploadFile, JSONResponse, auth middleware va Dockerda yengil

	ishlash.
Pandas/OpenPyXL	Analytics tablelarni o‘qish, filter qilish va Excel report export qilish uchun.
ReportLab	Executive PDF reportlarni backenddan generatsiya qilish uchun.
React + Vite	Dashboard UI tez build bo‘ladi, componentlar orqali sahifalar boshqariladi, lazy load bilan tezlik oshirilgan.
Recharts	KPI, chart, trend va analytics visualization uchun React bilan mos.
Docker Compose + Caddy	API, frontend, HTTPS reverse proxy va YouTube helper provider’ni bitta deploy stackda boshqarish uchun.
OpenAI API	Chatbotda kamera bo‘yicha operational savollar lokal javoblanadi, erkin savollar uchun OpenAI key qo‘yilganda javob sifati oshadi.

Nima uchun YOLO eng mos?

Bu loyiha real-time kuzatuvga yaqin ishlaydi. YOLO oilasi tezlik/aniqlik balansida yaxshi: yolov8n/yolov8s yengil, YOLO11m fine-tuning uchun kuchliroq. Faster R-CNN kabi model aniqroq bo‘lishi mumkin, lekin real-time live stream uchun og‘irroq. RT-DETR ham qo‘llab-quvvatlanadi, lekin default pipeline YOLO bilan sodda va barqaror.

7. Fine-tuning nima va bizda qayerda ishlaydi

Fine-tuning - oldindan o‘qitilgan modelni o‘z datasetimizga moslab qayta o‘qitish. Masalan, ustoz talab qilganidek ko‘cha kameralaridan screenshot/frame yig‘iladi, odam va obyektlar label qilinadi, model shu muhitga moslashadi. Natijada trainingdan chiqqan best.pt fayl custom model bo‘ladi.

10. Dataset yig‘ish: scripts/collect_images.py orqali live/video manbalardan taxminan 300 ta frame saqlash.
11. Label qilish: Roboflow ishlatilmasa, local label tool yoki autolabel + manual correction ishlatiladi.
12. Split: train/val/test taxminan 70%/20%/10%.
13. Augmentation: blur, brightness, contrast, noise kabi holatlar modelni real sharoitga moslaydi.
14. Training: scripts/train_yolo11m.py YOLO11m yoki mos modelni datasetga fine-tune qiladi.
15. Natija: runs/detect/train/weights/best.pt hosil bo‘ladi.
16. Deploy: Fine-tuning sahifasida best.pt upload qilinadi, aktiv model qilib tanlanadi.
17. Ishlatish: detector restart qilinganda main_detector.py tanlangan .pt model bilan detection qiladi.

Savol	Javob
Fine-tuning chatbotdami?	Yo‘q, bu YOLO detection model uchun. Chatbot OpenAI API yoki lokal fallback bilan ishlaydi.
best.pt nima?	Training jarayonida validation natijasi eng yaxshi chiqqan YOLO model weight fayli.
Nima uchun kerak?	Oddiy pretrained YOLO umumiy objectlarni taniydi. Custom camera muhitida odamlar, burchak, yorug‘lik va object ko‘rinishi boshqacha bo‘lsa, fine-tuning aniqlikni oshiradi.
Qayerdan test qilinadi?	FineTuning.tsx sahifasi /api/fine-tuning/test-image endpointiga rasm yuboradi va annotated output qaytaradi.

8. Ishga tushirish va deploy

Local ishga tushirish

```
python3 -m venv .venv
source .venv/bin/activate
```

```
pip install -r requirements.txt
uvicorn app.api_server:app --host 0.0.0.0 --port 8000 --reload
```

```
cd frontend
npm install
npm run dev
```

Detector namunasi

```
python app/main_detector.py --url "rtsp://user:pass@192.168.1.132:554/" --camera-id cam_01
--site "Entrance" --model yolov8n.pt --speed-mode fast --width 640 --height 360
```

AWS/Docker deploy

```
git pull origin main
docker compose up -d --build
```

Deploy eslatmasi

Docker API image yengil server uchun requirements-api.txt bilan quriladi. To‘liq YOLO/PyTorch detector uchun kuchliroq server yoki lokal Mac detector/relay ishlatiladi. Model weight fayllari GitHubga commit qilinmaydi, models/ papkaga alohida yuklanadi.

9. Muammolar va tezkor yechimlar

Muammo	Sabab	Yechim
RTSP ishlamayapti	Kamera lokal IP’da, detector boshqa Wi-Fi/serverda.	Detectorni kamera bilan bir LAN’da ishga tushiring yoki VPN/port forwarding sozlang.
YouTube Original bor, Detection qora	Detection relay frame yubormayapti yoki has_frame false.	auto_relay_manager ishlayotganini tekshiring, /api/relay/cameras da has_frame true bo‘lishi kerak.
Live 2 sekund kechikyapti	YOLO inference + frame upload + browser polling latency.	fast mode, kichik model, 640x360, detect-every kattaroq, duplicate kamera processlarini o‘chirish.
2-3 odam birga yursa object deb sanayapti	Person boxlarga yopishgan classlar false-positive bo‘lishi mumkin.	main_detector.py dagi attached object filtering va confidence thresholdni sozlash.
Chatbot hamma savolga javob bermayapti	OpenAI key yo‘q yoki savol lokal fallback doirasidan tashqarida.	Settings -> OpenAI API key qo‘yish, operational savollar uchun fallbackni kengaytirish.
Executive report ishlamayapti	Report endpoint/auth/data yoki ReportLab export xatosi.	/api/reports/executive/pdf va backend loglarni tekshirish.
Fine-tuned model ishlamayapti	Noto‘g‘ri fayl yoki detector hali eski model bilan yurgan.	Faqat .pt yuklang, select qiling, detector/relay restart qiling.

10. Og‘zaki himoya uchun tayyor qisqa javoblar

Savol	Qisqa javob
Loyiha nima qiladi?	Kameralardan video olib, YOLO bilan odam/obyektlarni aniqlaydi, risk hisoblaydi va dashboardlarda real-time ko‘rsatadi.
Backend qayerda?	app/api_server.py asosiy backend; FastAPI endpointlar shu yerda.
Frontend qayerda?	frontend/src/app/components/pages ichida dashboard sahifalari bor.
YOLO nima uchun?	Tez, real-timega mos, pretrained modeli bor, fine-tuning oson va .pt formatda deploy qilish qulay.
OpenCV nima uchun?	Video streamdan frame olish, resize, annotate, JPEG saqlash va

	RTSP/MP4/webcam ishlatish uchun.
Fine-tuning nima?	Pretrained YOLO modelni oʻz datasetimizga moslab qayta oʻqitish; natijasi best.pt.
Chatbot nimada ishlaydi?	Operational kamera savollari lokal fallback; OpenAI key boʻlsa erkin savollar OpenAI orqali.
Data flow qanday?	Camera/video -> OpenCV frame -> YOLO detection -> database/frames -> FastAPI -> React dashboard.

11. Foydali fayllar roʻyxati

- README.md - local setup va asosiy start commandlar.
- DEPLOY.md - Docker/VPS deploy tartibi.
- docs/local_no_roboflow_training.md - Roboflow ishlatmasdan local training yoʻli.
- docs/fine_tuning_exam_pipeline.md - fine-tuning exam pipeline.
- scripts/collect_images.py - dataset frame yigʻish.
- scripts/train_yolo11m.py - YOLO11m training.
- frontend/src/app/components/pages/FineTuning.tsx - model upload/test UI.
- app/api_server.py lines around /api/fine-tuning/* - model registry va test endpointlari.